

## บทที่ 3

# หลักการเขียนโปรแกรมแบบซีควนเชียลและโปรแกรมแบบขนาน

### 3.1 หลักการของการเขียนโปรแกรมแบบซีควนเชียล

การเขียนโปรแกรม ในลักษณะซีควนเชียลมีมานานแล้ว ตั้งแต่ที่ผู้ต้องการสร้างแอปพลิเคชัน โปรแกรมบนเครื่องเครื่องหนึ่ง ผู้ใช้ก็มีการเขียนซอร์สโค้ดโปรแกรมขึ้นบนเครื่องนั้น และทำการคอมไพล์โปรแกรมเพื่อให้สามารถใช้งานได้ โดยเมื่อต้องการจะพัฒนาโปรแกรมใหม่ก็จำเป็นต้องมีอัลกอริทึมของตัวโปรแกรมเดิม ไม่อย่างนั้นก็จำเป็นต้องมีการใช้เครื่องมือ(Tool) ต่างๆมาช่วย สาเหตุที่ผู้ใช้มักเขียนโปรแกรมในลักษณะซีควนเชียลเนื่องจาก

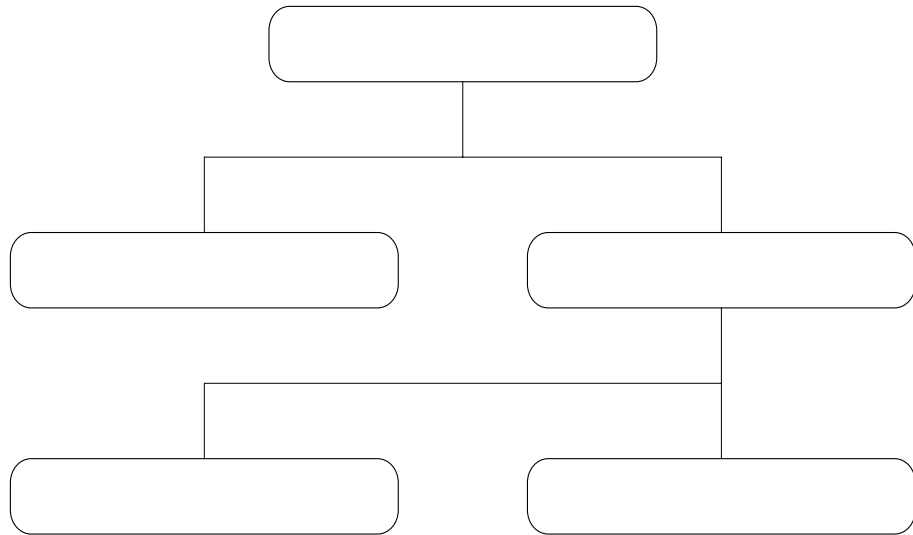
- เนื่องจากว่ามีรูปแบบอัลกอริทึมต่างมากมายหลายแบบซึ่งได้คิดค้นมานานแล้วและช่วยผู้ใช้งานสะดวกในการออกแบบโปรแกรม
- เนื่องจากการใช้แมชชีนโมเดลรูปแบบเดียวกัน ซึ่งสามารถรองรับได้กับทุกรูปแบบอัลกอริทึม และทุกๆแพลตฟอร์ม
- เนื่องจากมีภาษาทางซีควนเชียลหลายตัวที่เป็นที่นิยมใช้กัน เช่น ซี, พอร์แทรน, โคบอล เป็นต้น

โอเปอเรชัน ที่สำคัญในการเขียนโปรแกรมแบบซีควนเชียลมี 3 อย่าง คือ

- ซีควนเชียล โอเปอเรชัน เป็นการเขียนคำสั่งตามลำดับของการประมวลผลจากบนลงล่าง
- คอนดิชัน โอเปอเรชันเป็นการเขียนตามเงื่อนไขต่างๆและประมวลผลตามเงื่อนไขที่ถูกเลือกนั้น
- อิเทอเรทีฟ โอเปอเรชันเป็นการเขียนตามคำสั่งให้ประมวลผลซ้ำกันภายในบล็อกคำสั่งหนึ่ง

### 3.2 หลักการของการเขียนโปรแกรมแบบขนาน

ในระบบซีควนเชียลคอมพิวเตอร์ จะมีเพียงหนึ่งโปรแกรมที่ทำการประมวลผลในเวลาที่กำหนดซึ่งคอมพิวเตอร์จะใช้งานเพียง 1 ซีพียู หรือ 1 โปรเซสเซอร์ในการทำงานแต่ละครั้ง ซึ่งโปรแกรมที่ถูกประมวลผลนี้เรียกว่า โปรเซส โปรเซสจะประกอบไปด้วยโปรแกรม ข้อมูล สตэк และระบบปฏิบัติการเป็นโครงสร้าง โดยแต่ละโปรเซสจะมีเพียงหนึ่งหมายเลขโปรเซส (process ID) ซึ่งเป็นค่าข้อมูลประเภทอินทิจอร์ โดยซีควนเชียลคอมพิวเตอร์สามารถทำมัลติโปรแกรมมิงได้เพียงกำหนดให้งานหนึ่งมีหลายๆโปรเซสและทำโปรเซสสลับกันไปมา แต่ก็ไม่ได้เป็นการทำงานที่เป็นแบบขนานที่แท้จริง เนื่องจากการประมวลผลใช้ตัวโปรเซสเซอร์แค่ตัวเดียว ซึ่งต่างจากแบบขนานที่มีโปรเซสเซอร์หลายตัวมาช่วยกันประมวลผล ซึ่งจากรูป 3.2.1 การเขียนโปรแกรมแบบขนานสามารถแบ่งได้เป็น 2 ประเภท คือ



Parallel

รูปภาพ 3-1 แสดงประเภทของ การเขียน โปรแกรมแบบขนาน

1. SIMD (Single Instruction Multiple Data) หมายถึง ระบบคอมพิวเตอร์แบบขนานที่แต่ละโพรเซสเซอร์สามารถประมวลผลคำสั่งเดียว โดยใช้ข้อมูลหลายๆตัวในเวลาเดียวกัน

SIMD

2. MIMD (Multiple Instruction Multiple Data) หมายถึง ระบบคอมพิวเตอร์แบบขนานที่แต่ละโพรเซสเซอร์สามารถประมวลผลหลายๆคำสั่ง ซึ่งอยู่บนข้อมูลต่างชนิดกันได้ ซึ่งเป็นการแบ่งงานออกเป็นส่วนๆและแต่ละส่วนจะถูกประมวลผลโดยโพรเซสเซอร์ขนาน ทำให้การประมวลผลเป็นไปอย่างมีประสิทธิภาพ

ในกประมวลผลโพรเซสแบบขนาน นี้จะให้แต่ละส่วนที่มีอยู่สามารถแลกเปลี่ยนข้อมูลกันใน ช่วงเวลาหนึ่งได้ ดังนั้นจึงมีการติดต่อสื่อสารระหว่างโพรเซส ซึ่งมีอยู่ 2 วิธี คือ

Message Passing

1. ใช้วิธีการส่งแมสเสจ (Message Passing) คือจะนำการเชื่อมต่อเครือข่ายมาใช้ในการแลกเปลี่ยนข้อมูลระหว่างโพรเซสเซอร์ ซึ่งได้แก่ MPI และ PVM เป็นต้น
2. ใช้วิธีการแชร์หน่วยความจำ (Share Memory) ซึ่งเมื่อประมวลผลโปรแกรมแบบขนานและตัวโพรเซสเซอร์จะต้องระบุถึงจำนวนโพรเซสที่ต้องการมีการประมวลผล และพร้อมที่จะส่งโพรเซสเหล่านี้ไปให้กับตัวโพรเซสเซอร์ที่ว่างอยู่ทำงานทันที

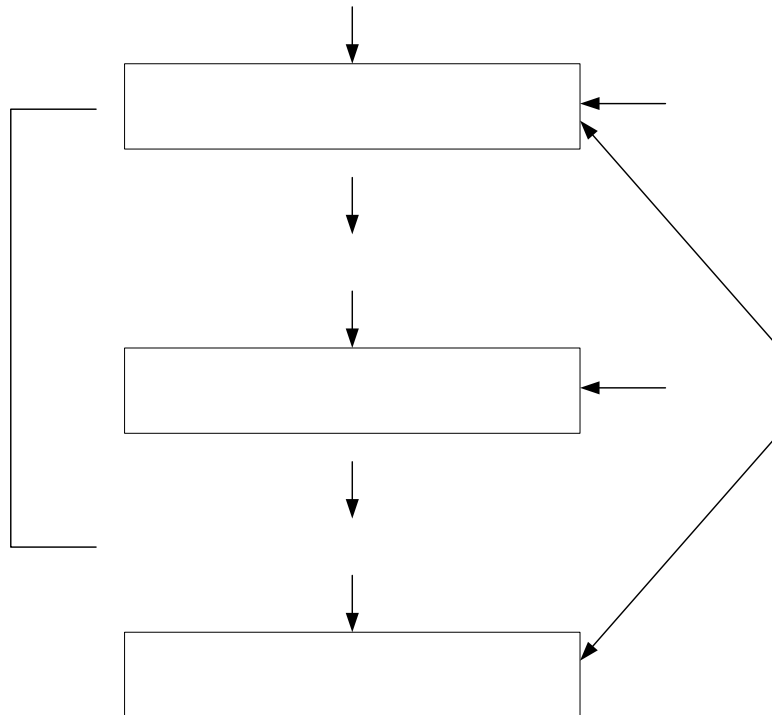
โดยส่วนมากการเขียนโปรแกรมแบบขนานนี้จะไม่สนับสนุนเครื่องมือ ต่างๆที่มีอยู่ และยังไม่เป็นที่เข้าใจหรือยอมรับจากผู้ใช้เป็นส่วนมาก โดยจะประกอบด้วยโมเดล 2 ระดับ คือ

1. โปรแกรมมิ่ง โมเดล เป็นโมเดลที่เกี่ยวข้องกับสิ่งที่โปรแกรมเมอร์มองเห็นและใช้ในการพัฒนา โปรแกรมแบบขนาน

2. เนทีฟ โมเดล เป็นโมเดลระดับต่ำกว่าระดับโปรแกรมมิ่ง โมเดล ซึ่งเป็นสิ่งที่โปรแกรมเมอร์มองเห็น ตามแต่ละแพลตฟอร์มของคอมพิวเตอร์แบบขนาน (parallel computer)

### 3.2.1 เอนไวรอนเมนต์ของการเขียนโปรแกรมแบบขนาน (Parallel Programming Environment)

จากมุมมองของผู้ใช้ ปกติระบบโครงสร้างของระบบประมวลผลแบบขนานเป็นไปดังรูปที่ 3-2 ซึ่งในตอนแรกจะนำส่วนของอัลกอริทึม ทั้งแบบ ซีควนเชียลหรือแบบขนานมาทำการคอมไพล์เมนต์ โดยใช้ส่วนของภาษาแบบขนาน หรือ ทูลต่างๆเข้าช่วยในการทำให้ได้ ซอร์สโปรแกรม ซึ่งจะผ่านส่วนของตัวคอมไพเลอร์ ซึ่งทำหน้าที่แปลง ซอร์สโค้ด ให้เป็นเนทีฟโค้ด(native code) ซึ่งจะอยู่ในลักษณะ โค้ดแบบขนาน (Parallel Code) เพื่อจะนำมาใช้กับแพลตฟอร์มที่เป็นแบบขนานได้ โดยในทุกขั้นตอนจะมีส่วนของรันไทม์ และ ไบเบรารี เพื่อใช้ในการเรียกโหนดรูทีนที่ได้อ้างอิงไว้ในตัวโค้ด



รูปภาพที่ 3-2 แสดงโครงสร้างของการเขียนโปรแกรมแบบขนาน (sequential or Parallel)

### 3.2.2 แนวทางการทำ Parallel Programming

ปัจจุบันมีรูปแบบโมเดลของการเขียนโปรแกรมแบบขนานอยู่ 4 รูปแบบที่นิยมใช้กัน คือ

1. Implicit
2. Data parallel

Usr (P

(Sequential or Pa

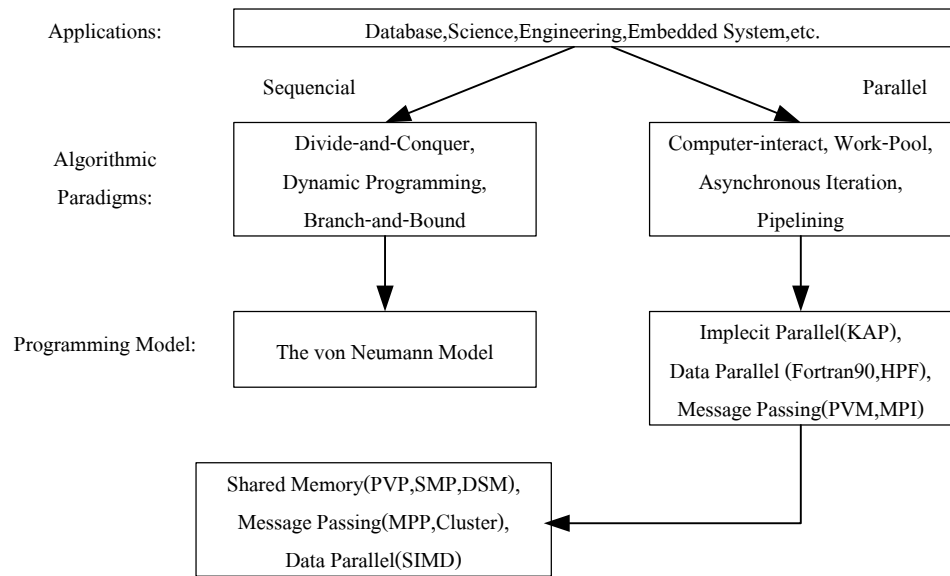
3. Message-passing
4. Shared variable

โดยโมเดลทั้งหลายนี้มักจะใช้กับภาษาฟอร์แทรนหรือซี เป็นส่วนขยายรูปแบบ โดยจะแบ่งได้เป็น 3 แนวทางคือ

1. ไลบรารีซับรูทีน นอกจากไลบรารีของโปรแกรมประเภท ซีควนเชียลโปรแกรม แล้วยังมีการเพิ่ม ไลบรารี ที่ทำให้สามารถประมวลผลคำสั่งแบบขนาน ยกตัวอย่างเช่น MPI, PVM โดยข้อดีของแนวทางประเภทนี้คือ ง่ายในการใช้งานโปรแกรม และไม่จำเป็นต้องมีการ อิมพลีเมนต์คอมไพเลอร์ใหม่ แต่มีข้อเสียคือ ไม่มีตรงส่วนของการตรวจสอบเวลาในการคอมไพล์ การตรวจสอบความถูกต้องหรือการออปติไมซ์ ซอร์สโค้ด
2. นิวคอนสตรัค แนวทางนี้จะต่างจากแนวทางแรกคือจะสร้างส่วนคอมไพเลอร์ขึ้นใหม่ให้มีการสนับสนุนการประมวลผลแบบขนาน โดยวิธีนี้มีข้อดีตรงที่เราจะได้ส่วนของการตรวจสอบความถูกต้องจากคอมไพเลอร์ การตรวจสอบเวลาการคอมไพล์ หรือการออปติไมซ์ ซอร์สโค้ด แต่มีข้อเสียที่จะต้องมีการสร้างคอมไพเลอร์ขึ้นใหม่ ทำให้ยากในการใช้โปรแกรมคอมไพเลอร์ใหม่
3. ไคเรกทีฟจะอยู่ระหว่างกลางของทั้งสองแนวทางที่กล่าวมา คือสามารถทำให้ใช้ซีควนเชียลโปรแกรมในรูปแบบที่กำหนดเพื่อให้กลายเป็นโปรแกรมแบบขนานได้ โดยใช้ตัวไคเรกทีฟเข้ามาแทรกในส่วนซีควนเชียลโปรแกรม โดยถ้านำเอาส่วนไคเรกทีฟออกก็สามารถใช้งานกับซีควนเชียลแพลตฟอร์มอื่นๆได้

### 3.3 การเปรียบเทียบระหว่างการเขียนโปรแกรมแบบซีควนเชียลกับโปรแกรมแบบขนาน

จากรูปที่ 3-3 จะเห็นได้ว่าการเขียนโปรแกรมแบบซีควนเชียลจะมีโมเดลอยู่โมเดลเดียว แต่แบบขนานจะมีอยู่สองโมเดล



รูปภาพที่ 3-3 การเปรียบเทียบระหว่างการเขียนโปรแกรมแบบซีควนเชียลกับแบบขนาน

จากตารางที่ 3-1 จะเป็นการเปรียบเทียบข้อดีข้อเสียของซีควนเชียลโปรแกรมและ โปรแกรมแบบขนาน

| รูปแบบ                | ข้อดี                                                                                              | ข้อเสีย                                                                   |
|-----------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| 1. ซีเควนเซียลโปรแกรม | 1. ผู้คนใช้งานเป็นจำนวนมาก เพราะคุ้นเคยกับการเขียนในรูปแบบนี้มาก่อน                                | 1. การประมวลผลทำได้ช้ากว่าโปรแกรมแบบขนานบนเครื่องที่มีการทำงานเป็นแบบขนาน |
|                       | 2. มีเครื่องมือ สนับสนุนอยู่เป็นจำนวนมาก                                                           | 2. ทำให้ตัวโปรแกรมเมอร์ต้องทำงานหนัก                                      |
|                       | 3. รูปแบบในการเขียนค่อนข้างง่าย                                                                    |                                                                           |
| 2. โปรแกรมแบบขนาน     | 1. เมื่อมีการประมวลผลในงานที่ต้องใช้ความเร็วโปรแกรมเมอร์สูงจะทำงานได้รวดเร็วกว่าซีเควนเซียลโปรแกรม | 1. ผู้คนมีความคุ้นเคยต่อรูปแบบการเขียนประเภทนี้น้อย                       |
|                       | 2. เป็นการใช้งานตัวโปรแกรมเมอร์ได้อย่างมีประสิทธิภาพ                                               | 2. มีทูลสนับสนุนอยู่น้อยมาก                                               |
|                       |                                                                                                    | 3. รูปแบบการเขียนค่อนข้างซับซ้อน                                          |

ตารางที่ 3-1 แสดงข้อเปรียบเทียบระหว่างซีเควนเซียลโปรแกรมและโปรแกรมแบบขนาน